



Fifty shades of ProVerif subtleties



Alexandre Debant

Université de Lorraine, Inria, CNRS, Nancy, France

25th years of ProVerif
June 3rd, 2026 - Paris, France

<https://www.wooclap.com>

Code d'événement

PHRFMR



Rules of the quizz

Number of questions: 10

Question format: what does ProVerif output?

`true`, `false`, `cannot be proved`

Verification summary:

Query `event(E) ==> event(F)` is `true`.

General comments:

1. **Very important:** there is no typo in the slides!
Truth is what is written, not what I might ambiguously say!
2. this talk is about ProVerif v2.05 (nothing about specific features of development branches)
3. always assume the following declarations:

```
free c_pub:channel.  
free c_priv: channel [private].  
const ok:bitstring.
```

<https://www.wooclap.com>



Code d'événement

PHRFMR

Protocol syntax

$P, Q := 0$
| new n ; P
| in(c, x); P
| out(c, u); P
| let $u = v$ in P else Q
| if b then P else Q
| (P | Q)
| ! P
| event $e(u_1, \dots, u_n)$; P

Protocol syntax

$P, Q := 0$
| new n ; P
| in(c, x); P
| out(c, u); P
| let $u = v$ in P else Q
| if b then P else Q
| ($P \mid Q$)
| ! P
| event $e(u_1, \dots, u_n)$; P

+

Restrictions:

$\rho := F_1 \ \&\& \ \dots F_n \ \&\& \Rightarrow H$



“Consider only traces that satisfy
 ρ , i.e. $tr \vdash \rho$ ”

Question 0

```
event Target.  
query event(Target).  
process  
    event Target
```

What does ProVerif output?

true

false

cannot be proved

Question 0

```
event Target.  
query event(Target).  
process  
    event Target
```

What does ProVerif output?

true
false
cannot be proved

FALSE

Verification summary:

Query `not event(Target)` is `false`.

Question 1

```
type key.  
fun enc(bitstring,key):bitstring.  
fun dec(bitstring,key):bitstring  
reduc forall m:bitstring, k:key;  
    dec(enc(m,k),k) = m.  
  
query event(Target).  
  
process  
    new k:key;  
    (  
        in(c_pub,att_ctxt:bitstring);  
        out(c_priv, dec(att_ctxt,k))  
    ) | (  
        in(c_priv,x_m:bitstring);  
        event Target  
    )
```

What does ProVerif output?

true

false

cannot be proved

Question 2

```
type key.  
fun enc(bitstring,key):bitstring.  
fun dec(bitstring,key):bitstring.  
equation forall m:bitstring, k:key;  
    dec(enc(m,k),k) = m.  
  
query event(Target).  
  
process  
    new k:key;  
    (  
        in(c_pub,att_ctxt:bitstring);  
        out(c_priv, dec(att_ctxt,k))  
    ) | (  
        in(c_priv,x_m:bitstring);  
        event Target  
    )  
)
```

What does ProVerif output?

true

false

cannot be proved

Question 3

```
fun enc(bitstring, key):bitstring.  
  reduc forall m:bitstring, k:key;  
    dec(enc(m, k), k) = m.  
  
const USE_ENC, NO_ENC:tag_t.  
  
query event(Target).  
  
process  
  new k:key;  
  Sender_WITHOUT_enc  
  | Sender_WITH_enc(k)  
  | (  
    in(c_priv, (tag:tag_t, x:bitstring));  
    if tag = USE_ENC then  
      let m = dec(x, k) in  
        (* ... *) 0  
    else  
      let m = x in  
        event Target  
  )
```

```
let Sender_WITHOUT_enc =  
  new n:bitstring;  
  out(c_priv, (NO_ENC, n)).  
  
let Sender_WITH_enc(k:key) =  
  new n:bitstring;  
  out(c_priv, (USE_ENC, enc(n, k))).
```

What does ProVerif output?

true
false
cannot be proved

Question 4

```
fun eq(bitstring,bitstring):bool
  reduc forall x:bitstring;
    eq(x,x) = true.

query event(Target).

process
  new a:bitstring;
  new b:bitstring;
  if eq(a,b) then
    (* ... *) 0
  else
    event Target
```

What does ProVerif output?

true

false

cannot be proved

Question 5

```
fun eq(bitstring,bitstring):bool
  reduc forall x:bitstring;
    eq(x,x) = true.

query event(Target).

process
  new a:bitstring;
  new b:bitstring;
  let b = eq(a,b) in
    (* ... *) 0
  else
    event Target
```

What does ProVerif output?

true
false
cannot be proved

Question 6

```
const YES,NO:bitstring.

event Then(bitstring).
event Else(bitstring).
event End(bitstring).

query event(Then(YES)).
query event(Else(NO)).
query event(End(YES)).
query event(End(NO)).

process
  in(c_pub,x:bitstring);
  if x = YES then (
    event Then(x)
  ) else (
    event Else(x)
  ) |
  event End(x)
```

Which events are reachable?

pick the correct ones (from 0 to 4)

Question 7

```
fun enc(bitstring, key): bitstring.  
  reduc forall m: bitstring, k: key;  
    dec(enc(m, k), k) = m.  
  
free my_secret: bitstring [private].  
query attacker(my_secret).  
  
process  
  new k: key;  
  ( (* output on an "authenticated" channel *)  
    out(c_priv, enc(my_secret, k));  
    out(c_pub, enc(my_secret, k))  
  ) | (  
    in(c_pub, x: bitstring);  
    let m = dec(x, k) in  
    out(c_pub, m)  
  )
```

What shall ProVerif output?

true
false

and what does it actually output?

true
false
cannot be proved

Question 8

```
(* non-randomized encryption scheme*)
type key.
fun enc(bitstring, key):bitstring.
  reduc forall m:bitstring, k:key;
    dec(enc(m, k), k) = m.

event StopExec(bitstring).
restriction x:bitstring; event(StopExec(x)).

process
  new k:key;

  (* strong secrecy of id *)
  ! in(c_pub, id:bitstring);
  new n:bitstring;

  let encrypted_id = enc(diff[id, n], k) in

  event StopExec(encrypted_id);
  out(c_pub, encrypted_id)
```

What does ProVerif output?

true
false
cannot be proved

Question 9

```
event E(bitstring,bitstring).
event F(bitstring,bitstring).
restriction xL,xR,yL,yR,xL2,xR2,yL2,yR2:bitstring;
    event(E(diff[xL,xR],diff[yL,yR]))
    && event(F(diff[xL2,xR2],diff[yL2,yR2])) ==> yR <> yR2.
(* Equivalent to:
- on the left: event(E(xL,yL)) && event(F(xL2,yL2)) ==> true,
    i.e., nothing special
- on the right: event(E(xR,yR)) && event(F(xR2,yR2)) ==> yR <> yR2,
    i.e., second argument must be different
*)

process
    in(c_pub,x:bitstring);
    in(c_pub,y:bitstring);

    event E(x,x);
    event F(x,y);

    out(c_pub, diff[x,x])
```

What does ProVerif output?

true

false

cannot be proved

Question 10

```
event StopExec.  
  
query event(StopExec).  
  
process  
  out(c_priv,9) |  
  !(  
    in(c_priv,n:nat);  
    if n - 2 < 1 then  
      event StopExec  
    else  
      out(c_priv, n-2)  
  )
```

What does ProVerif output?

true

false

cannot be proved

Question bonus

```
event Target(nat).  
query event(Target(0)).  
  
process  
  counter <- 5;  
  !(  
    counter <- counter - 1;  
    event Target(counter)  
  )
```

What does ProVerif output?

true
false
cannot be proved
invalid syntax

Question 1

```
type key.  
fun enc(bitstring,key):bitstring.  
fun dec(bitstring,key):bitstring  
reduc forall m:bitstring, k:key;  
    dec(enc(m,k),k) = m.  
  
query event(Target).  
  
process  
    new k:key;  
    (  
        in(c_pub,att_ctxt:bitstring);  
        out(c_priv, dec(att_ctxt,k))  
    ) | (  
        in(c_priv,x_m:bitstring);  
        event Target  
    )
```

What does ProVerif output?

true

false

cannot be proved

Question 1

```
type key.  
fun enc(bitstring,key):bitstring.  
fun dec(bitstring,key):bitstring  
reduc forall m:bitstring, k:key;  
    dec(enc(m,k),k) = m.  
  
query event(Target).  
  
process  
    new k:key;  
    (  
        in(c_pub,att_ctxt:bitstring);  
        out(c_priv, dec(att_ctxt,k))  
    ) | (  
        in(c_priv,x_m:bitstring);  
        event Target  
    )  
)
```

What does ProVerif output?

true

false

cannot be proved

TRUE

Question 2

```
type key.  
fun enc(bitstring,key):bitstring.  
fun dec(bitstring,key):bitstring.  
equation forall m:bitstring, k:key;  
    dec(enc(m,k),k) = m.  
  
query event(Target).  
  
process  
    new k:key;  
    (  
        in(c_pub,att_ctxt:bitstring);  
        out(c_priv, dec(att_ctxt,k))  
    ) | (  
        in(c_priv,x_m:bitstring);  
        event Target  
    )
```

What does ProVerif output?

true

false

cannot be proved

Question 2

```
type key.  
fun enc(bitstring,key):bitstring.  
fun dec(bitstring,key):bitstring.  
equation forall m:bitstring, k:key;  
    dec(enc(m,k),k) = m.  
  
query event(Target).  
  
process  
    new k:key;  
    (  
        in(c_pub,att_ctxt:bitstring);  
        out(c_priv, dec(att_ctxt,k))  
    ) | (  
        in(c_priv,x_m:bitstring);  
        event Target  
    )  
)
```

What does ProVerif output?

true
false
cannot be proved

FALSE

Question 3

```
fun enc(bitstring, key):bitstring.  
  reduc forall m:bitstring, k:key;  
    dec(enc(m, k), k) = m.  
  
const USE_ENC, NO_ENC:tag_t.  
  
query event(Target).  
  
process  
  new k:key;  
  Sender_WITHOUT_enc  
  | Sender_WITH_enc(k)  
  | (  
    in(c_priv, (tag:tag_t, x:bitstring));  
    if tag = USE_ENC then  
      let m = dec(x, k) in  
        (* ... *) 0  
    else  
      let m = x in  
        event Target  
  )
```

```
let Sender_WITHOUT_enc =  
  new n:bitstring;  
  out(c_priv, (NO_ENC, n)).  
  
let Sender_WITH_enc(k:key) =  
  new n:bitstring;  
  out(c_priv, (USE_ENC, enc(n, k))).
```

What does ProVerif output?

true
false
cannot be proved

Question 3

```
fun enc(bitstring, key):bitstring.  
  reduc forall m:bitstring, k:key;  
    dec(enc(m, k), k) = m.  
  
const USE_ENC, NO_ENC:tag_t.  
  
query event(Target).  
  
process  
  new k:key;  
  Sender_WITHOUT_enc  
  | Sender_WITH_enc(k)  
  | (  
    in(c_priv, (tag:tag_t, x:bitstring));  
    if tag = USE_ENC then  
      let m = dec(x, k) in  
        (* ... *) 0  
    else  
      let m = x in  
        event Target  
  )
```

```
let Sender_WITHOUT_enc =  
  new n:bitstring;  
  out(c_priv, (NO_ENC, n)).  
  
let Sender_WITH_enc(k:key) =  
  new n:bitstring;  
  out(c_priv, (USE_ENC, enc(n, k))).
```

What does ProVerif output?

true
false
cannot be proved

TRUE

Question 4

```
fun eq(bitstring,bitstring):bool
  reduc forall x:bitstring;
    eq(x,x) = true.

query event(Target).

process
  new a:bitstring;
  new b:bitstring;
  if eq(a,b) then
    (* ... *) 0
  else
    event Target
```

What does ProVerif output?

true
false
cannot be proved

Question 4

```
fun eq(bitstring,bitstring):bool
  reduc forall x:bitstring;
    eq(x,x) = true.

query event(Target).

process
  new a:bitstring;
  new b:bitstring;
  if eq(a,b) then
    (* ... *) 0
  else
    event Target
```

What does ProVerif output?

true

false

cannot be proved

TRUE

Question 5

```
fun eq(bitstring,bitstring):bool
  reduc forall x:bitstring;
    eq(x,x) = true.

query event(Target).

process
  new a:bitstring;
  new b:bitstring;
  let b = eq(a,b) in
    (* ... *) 0
  else
    event Target
```

What does ProVerif output?

true
false
cannot be proved

Question 5

```
fun eq(bitstring,bitstring):bool
  reduc forall x:bitstring;
    eq(x,x) = true.

query event(Target).

process
  new a:bitstring;
  new b:bitstring;
  let b = eq(a,b) in
    (* ... *) 0
  else
    event Target
```

What does ProVerif output?

true
false
cannot be proved

FALSE

Question 6

```
const YES,NO:bitstring.

event Then(bitstring).
event Else(bitstring).
event End(bitstring).

query event(Then(YES)).
query event(Else(NO)).
query event(End(YES)).
query event(End(NO)).

process
  in(c_pub,x:bitstring);
  if x = YES then (
    event Then(x)
  ) else (
    event Else(x)
  ) |
  event End(x)
```

Which events are reachable?

pick the correct ones (from 0 to 4)

Question 6

```
const YES,NO:bitstring.  
  
event Then(bitstring).  
event Else(bitstring).  
event End(bitstring).  
  
query event(Then(YES)).  
query event(Else(NO)).  
query event(End(YES)).  
query event(End(NO)).  
  
process  
  in(c_pub,x:bitstring);  
  if x = YES then (  
    event Then(x)  
  ) else (  
    event Else(x)  
  ) |  
  event End(x)
```

Which events are reachable?

pick the correct ones (from 0 to 4)



Question 7

```
fun enc(bitstring, key): bitstring.  
  reduc forall m: bitstring, k: key;  
    dec(enc(m, k), k) = m.  
  
free my_secret: bitstring [private].  
query attacker(my_secret).  
  
process  
  new k: key;  
  ( (* output on an "authenticated" channel *)  
    out(c_priv, enc(my_secret, k));  
    out(c_pub, enc(my_secret, k))  
  ) | (  
    in(c_pub, x: bitstring);  
    let m = dec(x, k) in  
    out(c_pub, m)  
  )
```

What shall ProVerif output?

true
false

and what does it actually output?

true
false
cannot be proved

Question 7

```
fun enc(bitstring, key): bitstring.  
  reduc forall m: bitstring, k: key;  
    dec(enc(m, k), k) = m.  
  
free my_secret: bitstring [private].  
query attacker(my_secret).  
  
process  
  new k: key;  
  ( (* output on an "authenticated" channel *)  
    out(c_priv, enc(my_secret, k));  
    out(c_pub, enc(my_secret, k))  
  ) | (  
    in(c_pub, x: bitstring);  
    let m = dec(x, k) in  
    out(c_pub, m)  
  )
```

What shall ProVerif output?

true
false

TRUE

and what does it actually output?

true
false
cannot be proved

Question 7

```
fun enc(bitstring, key):bitstring.  
  reduc forall m:bitstring, k:key;  
    dec(enc(m, k), k) = m.  
  
free my_secret:bitstring [private].  
query attacker(my_secret).  
  
process  
  new k:key;  
  ( (* output on an "authenticated" channel *)  
    out(c_priv, enc(my_secret, k));  
    out(c_pub, enc(my_secret, k))  
  ) | (  
    in(c_pub, x:bitstring);  
    let m = dec(x, k) in  
    out(c_pub, m)  
  )
```

What shall ProVerif output?

true
false

TRUE

and what does it actually output?

true
false
cannot be proved

**CANNOT
BE PROVED**

Question 8

```
(* non-randomized encryption scheme*)
type key.
fun enc(bitstring, key):bitstring.
  reduc forall m:bitstring, k:key;
    dec(enc(m, k), k) = m.

event StopExec(bitstring).
restriction x:bitstring; event(StopExec(x)).

process
  new k:key;

  (* strong secrecy of id *)
  ! in(c_pub, id:bitstring);
  new n:bitstring;

  let encrypted_id = enc(diff[id, n], k) in

  event StopExec(encrypted_id);
  out(c_pub, encrypted_id)
```

What does ProVerif output?

true
false
cannot be proved

Question 8

```
(* non-randomized encryption scheme*)
type key.
fun enc(bitstring, key):bitstring.
  reduc forall m:bitstring, k:key;
    dec(enc(m, k), k) = m.

event StopExec(bitstring).
restriction x:bitstring; event(StopExec(x)).

process
  new k:key;

  (* strong secrecy of id *)
  ! in(c_pub, id:bitstring);
  new n:bitstring;

  let encrypted_id = enc(diff[id, n], k) in

  event StopExec(encrypted_id);
  out(c_pub, encrypted_id)
```

What does ProVerif output?

true
false
cannot be proved

FALSE

Question 8

```
(* non-randomized encryption scheme*)
type key.
fun enc(bitstring, key):bitstring.
  reduc forall m:bitstring, k:key;
    dec(enc(m, k), k) = m.

event StopExec(bitstring).
restriction x:bitstring; event(StopExec(x)).

process
  new k:key;

  (* strong secrecy of id *)
  ! in(c_pub, id:bitstring);
  new n:bitstring;

  let encrypted_id =

  event StopExec(enc
  out(c_pub, encrypted_id)
```

What does ProVerif output?

true
false
cannot be proved

FALSE

Verification summary:

Query(ies):

– Diff-equivalence is **false**.

Associated restriction(s):

– Restriction **not** event(StopExec(x)) encoded as **not** event2(StopExec(x), StopExec(x)) in bprocess 0.

Question 9

```
event E(bitstring,bitstring).
event F(bitstring,bitstring).
restriction xL,xR,yL,yR,xL2,xR2,yL2,yR2:bitstring;
    event(E(diff[xL,xR],diff[yL,yR]))
    && event(F(diff[xL2,xR2],diff[yL2,yR2])) ==> yR <> yR2.
(* Equivalent to:
- on the left: event(E(xL,yL)) && event(F(xL2,yL2)) ==> true,
    i.e., nothing special
- on the right: event(E(xR,yR)) && event(F(xR2,yR2)) ==> yR <> yR2,
    i.e., second argument must be different
*)

process
    in(c_pub,x:bitstring);
    in(c_pub,y:bitstring);

    event E(x,x);
    event F(x,y);

    out(c_pub, diff[x,x])
```

What does ProVerif output?

true

false

cannot be proved

Question 9

```
event E(bitstring,bitstring).
event F(bitstring,bitstring).
restriction xL,xR,yL,yR,xL2,xR2,yL2,yR2:bitstring;
    event(E(diff[xL,xR],diff[yL,yR]))
    && event(F(diff[xL2,xR2],diff[yL2,yR2])) ==> yR <> yR2.
(* Equivalent to:
- on the left: event(E(xL,yL)) && event(F(xL2,yL2)) ==> true,
    i.e., nothing special
- on the right: event(E(xR,yR)) && event(F(xR2,yR2)) ==> yR <> yR2,
    i.e., second argument must be different
*)

process
    in(c_pub,x:bitstring);
    in(c_pub,y:bitstring);

    event E(x,x);
    event F(x,y);

    out(c_pub, diff[x,x])
```

What does ProVerif output?

true

false

cannot be proved

TRUE

Question 9

```

event E(bitstring, bitstring)
event F(bitstring, bitstring)
restriction xL, xR, yL, yR, ...
    event(E(diff[xL, xR], ...))
    && event(F(diff[xL2, ...]))
(* Equivalent to:
- on the left: event(E(xL, ...))
  i.e., not ...
- on the right: event(E(xR, ...))
  i.e., ...
*)

```

```

process
  in(c_pub, bitstring)
  in(c_pub, bitstring)
  event E(x, ...);
  event F(x, y);
  out(c_pub, diff[x, x])

```



```

Biprocess 0 (that is, the initial process):
{1}in(c_pub, x: bitstring);
{2}in(c_pub, y: bitstring);
{3}event E(x,x);
{4}event F(x,y);
{5}out(c_pub, x)

-- Restriction event(E(choice[xL,xR],choice[yL,yR])) && event(F(choice[xL2,xR2],choice[yL2,yR2])) ==>
.
-- Diff-equivalence in biprocess 0.
Translating the process into Horn clauses...
Termination warning: v ≠ v_1 && attacker2(v_2,v) && attacker2(v_2,v_1) -> bad
Selecting 0
Termination warning: v ≠ v_1 && attacker2(v,v_2) && attacker2(v_1,v_2) -> bad
Selecting 0
Completing...
Termination warning: v ≠ v_1 && attacker2(v_2,v) && attacker2(v_2,v_1) -> bad
Selecting 0
Termination warning: v ≠ v_1 && attacker2(v,v_2) && attacker2(v_1,v_2) -> bad
Selecting 0
RESULT Diff-equivalence is true.

-----
Verification summary:
Query(ies):
- Diff-equivalence is true.
Associated restriction(s):
- Restriction event(E(choice[xL,xR],choice[yL,yR])) && event(F(choice[xL2,xR2],choice[yL2,yR2])) ==>
.
-----

```

if output?

proved



Question 10

```
event StopExec.  
  
query event(StopExec).  
  
process  
  out(c_priv,9) |  
  !(  
    in(c_priv,n:nat);  
    if n - 2 < 1 then  
      event StopExec  
    else  
      out(c_priv, n-2)  
  )
```

What does ProVerif output?

true

false

cannot be proved

Question 10

```
event StopExec.  
  
query event(StopExec).  
  
process  
  out(c_priv,9) |  
  !(  
    in(c_priv,n:nat);  
    if n - 2 < 1 then  
      event StopExec  
    else  
      out(c_priv, n-2)  
  )
```

What does ProVerif output?

true

false

cannot be proved

TRUE

Question bonus

```
event Target(nat).  
  
query event(Target(0)).  
  
process  
  counter <- 5;  
  !(  
    counter <- counter - 1;  
    event Target(counter)  
  )
```

What does ProVerif output?

true
false
cannot be proved
invalid syntax

Question bonus

```
event Target(nat).  
query event(Target(0)).  
  
process  
  counter <- 5;  
  !(  
    counter <- counter - 1;  
    event Target(counter)  
  )
```

What does ProVerif output?

true
false
cannot be proved
invalid syntax

TRUE

`counter <- counter - 1;` $\approx$ `let counter = counter - 1 in`

Who's the winner?



With Vincent, we are (trying to) writing a user-book on ProVerif
If you have other examples do not hesitate to share with us!